

Scientific computing and data visualization in Python

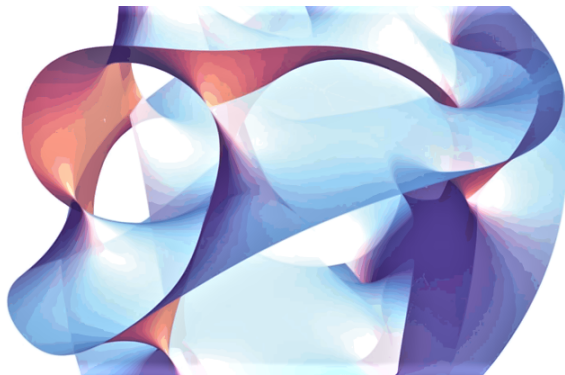
Adrian Kania¹

¹Department of Computational Biophysics and Bioinformatics

2026

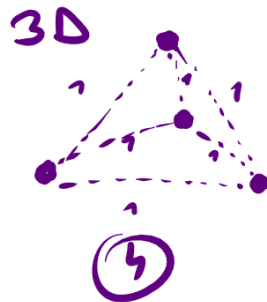
Higher dimensions peculiarities

Before the analysis, if our dataset consists of many many features (for example: several hundred, several thousand, several million... like images) they should be reduced to a smaller amount. Why? Firstly, to prevent overfitting - more parameters, worse generalization (in general). Moreover, it appears that in high-dimensional spaces there are some strange behaviors which do not occur in low-dimensional spaces (1D, 2D, 3D...). It is called **curse of dimensionality**.



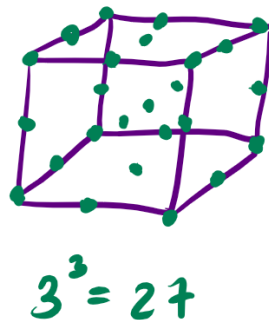
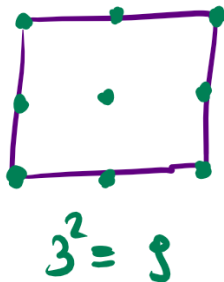
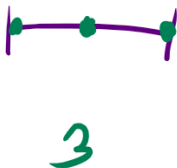
- What is the maximum number of points in 1D, 2D, 3D, ... space where distance between every pair of points is 1?

Higher dimensions peculiarities



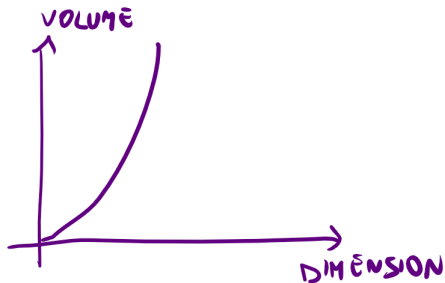
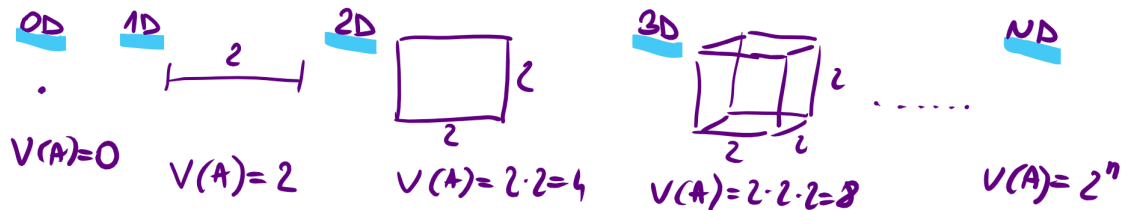
In 1D there are 2 points, in 2D there are 3 points,..., in ND there are $N + 1$ points.

Higher dimensions peculiarities



The higher dimension, more points is needed to fulfill a given subspace - there is more free space.

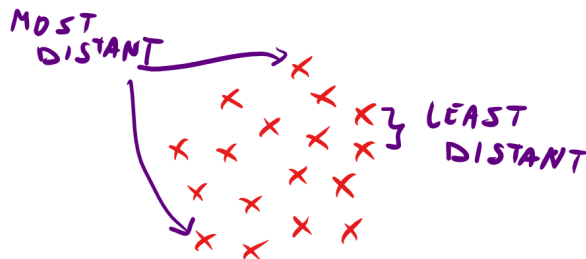
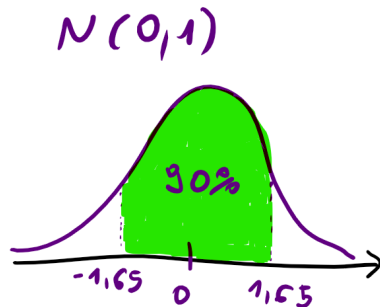
Higher dimensions peculiarities



It appears that volume increases very rapidly (for hypercube).

Higher dimensions peculiarities

Other facts



- In 1D, most of the volume is near the centre; in high-dimensional spaces most of the volume is in the "tail" (gaussian distribution).
- If the dimension increases, the distances between the most and least distant points converge (problematic for k-means)

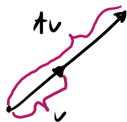
Eigenvalues and eigenvectors

Given n by n matrix A and nonzero vector v . Suppose there is λ which satisfies

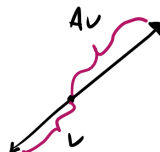
$$Av = \lambda v$$

v is called an **eigenvector** of A , λ is called an **eigenvalue** of A . Linear transformation A maps an eigenvector v in a simple way. Magnitude changes by λ , direction

IF $\lambda > 0$



IF $\lambda < 0$



Property

- If A is real and symmetric, then all eigenvalues are real (not complex).

```
from numpy import linalg as LA

A = np.array([[1,2],
              [5,6]])
lam, v = LA.eig(A)

print(lam) #eigenvalues
print()
print(v) #eigenvectors (columns)
```

```
[-0.53112887  7.53112887]
[[-0.79402877 -0.2928046 ]
 [ 0.60788018 -0.9561723 ]]
```

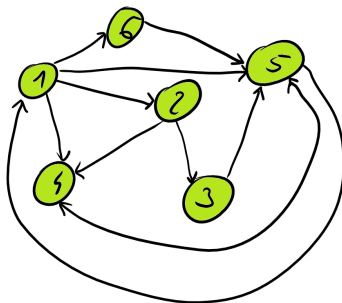
```
print(np.dot(A,v[:,0])) Av1
print()
print(lam[0]*v[:,0]) l1*v1
```

```
[ 0.4217316 -0.32286272]
[ 0.4217316 -0.32286272]
```

PageRank - Google Search algorithm as an example of eigenvalues and eigenvectors application

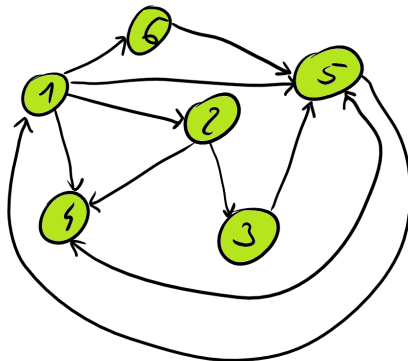
Let's say we have 6 web pages - 1,2,3,4,5, and 6 (all refers to a given topic, issue). There are the following links (references) between them:

- Page1 recommends: 2,4,5,6.
- Page2 recommends: 3,4
- Page3 recommends: 5
- Page4 recommends: 5
- Page5 recommends: 1,4.
- Page6 recommends: 5.



Question: What is the order the pages should be returned? Which seems the best?

PageRank - Google Search algorithm



which correspond to the following probability matrix (columns are references probabilities)

```

import numpy as np
A = np.array([[0,0,0,0,0.5,0],[0.25,0,0,0,0,0],[0,0.5,0,0,0,0],[0.25,0.5,0,0,0.5,0],[0.25,0,1,1,0,1],[0.25,0,0,0,0,0]])
A
array([[0. , 0. , 0. , 0. , 0.5 , 0. ],
       [0.25, 0. , 0. , 0. , 0. , 0. ],
       [0. , 0.5 , 0. , 0. , 0. , 0. ],
       [0.25, 0.5 , 0. , 0. , 0.5 , 0. ],
       [0.25, 0. , 1. , 1. , 0. , 1. ],
       [0.25, 0. , 0. , 0. , 0. , 0. ]])
  
```

PageRank - Google Search algorithm

```
import numpy as np
from numpy import linalg as LA
```

```
A = np.array([[0,0,0,0,0.5,0],[0.25,0,0,0,0,0],[0,0.5,0,0,0,0],[0.25,0.5,0,0,0.5,0],[0.25,0,1,1,0,1],[0.25,0,0,0,0,0]])
lam, v = LA.eig(A)
```

```
print(lam)
```

```
import matplotlib.pyplot as plt
```

```
#polar coordinates
```

```
def polar(z):
    a= z.real #x coordinate
    b= z.imag #y coordinate
    r = np.sqrt(a**2+b**2) #radius
    theta = np.arctan2(b,a) #angle
    return r,theta
```

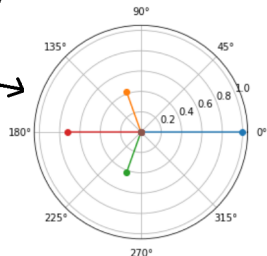
```
for x in lam:
    plt.polar([0,polar(x)[1]],[0,polar(x)[0]],marker='o')
```

```
print()
print(np.abs(lam)) #we take the highest eigenvalue
print()
print(np.abs(v[:,0])) #... and its corresponding eigenvector
```

```
[ 1.00000000e+00+0.j          -1.39845075e-01+0.39240258j
 -1.39845075e-01-0.39240258j   -7.20309850e-01+0.j
  6.27089983e-17+0.j          -2.02451367e-17+0.j          ]

[1.00000000e+00  4.16577040e-01  4.16577040e-01  7.20309850e-01
  6.27089983e-17  2.02451367e-17]

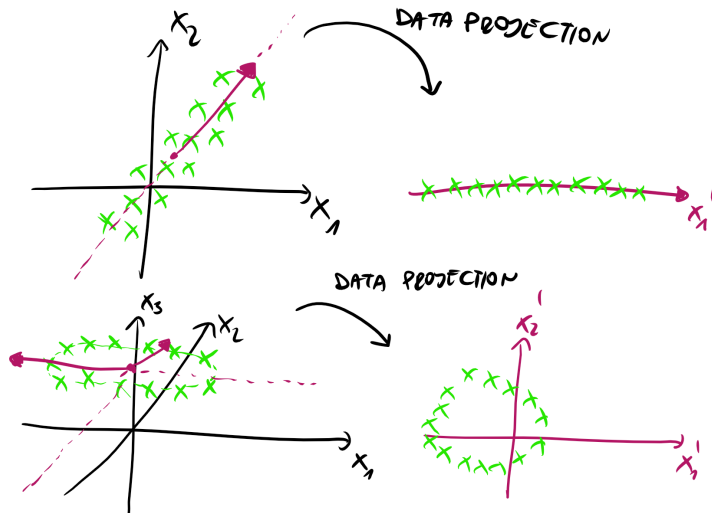
[0.37712362  0.0942809   0.04714045  0.51854497  0.75424723  0.0942809 ]
```



Taking the eigenvector corresponding the highest eigenvalue we have the ranking.

PCA (Principal component analysis) - main concepts

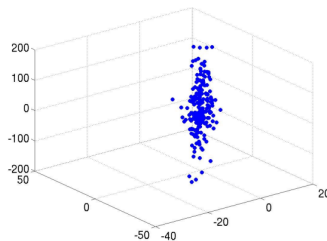
Main idea: seek most accurate data representation in a lower dimensional space.



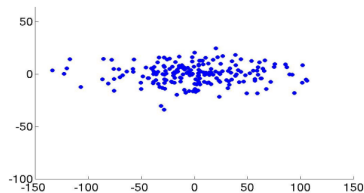
Technically, we're looking for orthogonal vectors in space, and then projecting the data onto those vectors in the direction of which the variance is greatest.

PCA - main concepts

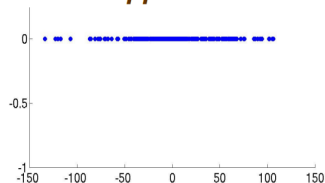
Main idea: seek most accurate data representation in a lower dimensional space.



best 2D approximation



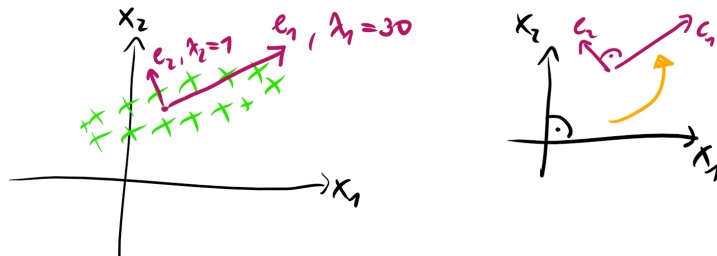
best 1D approximation



PCA preserves largest variances in the data.

PCA - how to find such vectors?

Fact: For a given symmetric matrix $M \in \mathbb{M}(n, n)$ there are n eigenvalues λ_i and corresponding eigenvectors e_i . Additionally, these vectors are orthogonal.



The higher λ_i value, the higher variance in e_i vector's direction. Interestingly, PCA can be thought of as finding new orthogonal basis by rotating the old axis until the directions of maximum variance are found.

Exercise: Generate 10000 data (X, Y) from 2D Gaussian distribution $N(\mu, \Sigma)$, where $\mu = [1, 2]$ and $\Sigma = \begin{bmatrix} 1 & 1.1 \\ 1.1 & 4 \end{bmatrix}$. Calculate the covariance matrix for (X, Y) and its eigenvalues and corresponding eigenvectors. Multiply the vectors by corresponding eigenvalues and plot both data and vectors (located in the mean of data).

PCA - theory behind

Data $D = \{x_1, x_2, \dots, x_n\}$. Each x_i is a d -dimensional vector. Let $e_1, e_2, \dots, e_d$ be all d eigenvectors of the covariance matrix S sorted in order of decreasing corresponding eigenvalue. Then, every data point x_i may be written as

$$x_i = \sum_{j=1}^d \alpha_j e_j = \alpha_1 e_1 + \dots + \alpha_k e_k + e_{k+1} + \dots + \alpha_d e_d$$

as there are d orthogonal vectors in $\mathbb{R}^d$ space - they are linearly independent and every other vector is a combination of them. We know that not all directions are equally important, so we may save only k first part. More precisely

$$\underbrace{\alpha_1 e_1 + \dots + \alpha_k e_k}_{\text{approximation}}$$

and

$$\underbrace{\alpha_{k+1} e_{k+1} + \dots + \alpha_d e_d}_{\text{error of approximation}}$$

and then $x_i \approx \alpha_1 e_1 + \dots + \alpha_k e_k$. The explained variance using first k vectors $e_1, e_2, \dots, e_k$ is $\frac{\lambda_1 + \lambda_2 + \dots + \lambda_k}{\lambda_1 + \lambda_2 + \dots + \lambda_d}$.

PCA - algorithm

Data $D = \{x_1, x_2, \dots, x_n\}$. Each x_i is a d -dimensional vector. Wish to use PCA do reduce dimension to k .

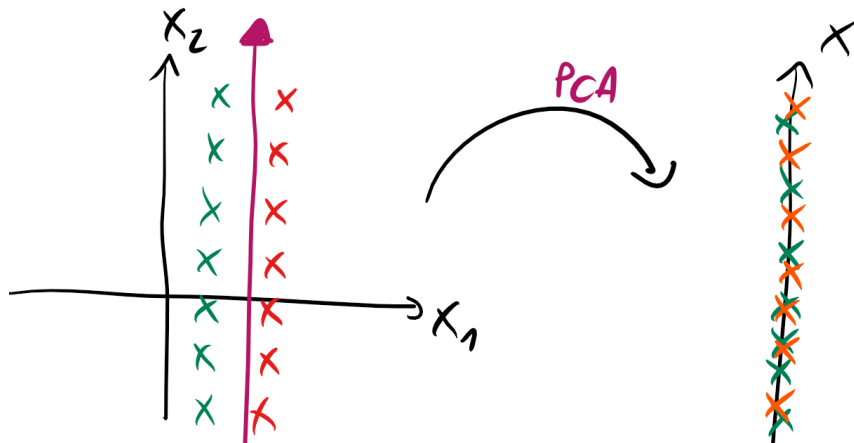
- Find the sample mean $\mu = \frac{1}{n} \sum_i x_i$,
- Substract sample mean from the data $z_i = x_i - \mu$,
- Compute the scatter matrix $S = \sum_i z_i z_i^T$,
- Compute eigenvectors $e_1, e_2, \dots, e_k$ corresponding to the k largest eigenvalues of S ,
- Let $e_1, e_2, \dots, e_k$ be the columns of matrix $E = [e_1, e_2, \dots, e_k]$.
- The desired y which is the closest approximation of x if $y = E^T z$.

Instead of steps 1,2,3 you may calculate the covariance matrix.

Exercise: Let $D = \{(1, 2), (2, 3), (3, 2), (4, 4), (5, 4), (6, 7), (7, 6), (9, 7)\}$. Plot the data and calculate 1D representation of the data. What is the explained variance there?

PCA - drawbacks

Notice PCA was designed for accurate data representation, not for data classification. It preserves as much variance in data as possible. Sometimes, the directions of maximum variance may be useless for classification.



ICA

- Vector x is a **linear combination** of vectors $x_1, x_2, \dots, x_n$ if there exist real numbers $\alpha_1, \alpha_2, \dots, \alpha_n \in \mathbb{R}$ such as:

$$x = \alpha_1 x_1 + \alpha_2 x_2 + \dots + \alpha_n x_n$$

- Vectors $x_1, x_2, \dots, x_n$ are **linearly dependent** if at least one of the vectors is a linear combination of remaining vectors.
- Vectors are **linearly independent** if they are not linearly dependent.

For a given set of features $f_1, f_2, \dots, f_n$, our aim is to find a another set features $f'_1, \dots, f'_k$, where $k \ll n$ where every feature f_i may be considered as a combination of $\{f'_i\}$ and $\{f'_i\}$ are linearly independent.

For example, in linear model, rather than modelling $y = \sum_i \alpha_i f_i$ we may restrict to $y = \sum_i \beta f'_i$ - less paramters to estimate, the same information as original features store.

ICA

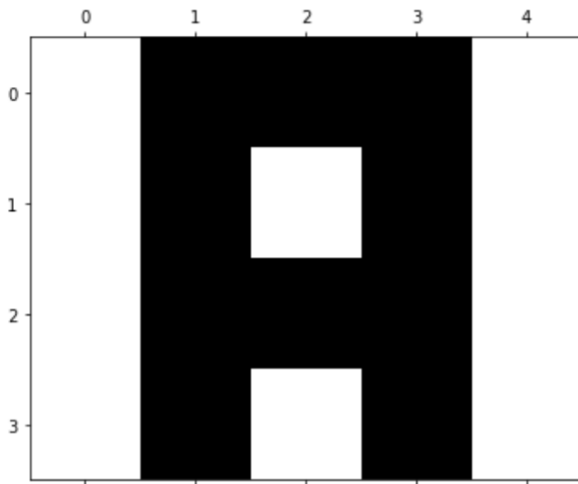
Images are stored as matrices. When we talk about black and white photos, there are only 0 and 1 in such a matrix.

```
A = np.array([[0,1,1,1,0],
               [0,1,0,1,0],
               [0,1,1,1,0],
               [0,1,0,1,0]])
```

```
print(A)
```

```
plt.matshow(A, cmap='Greys')
plt.show()
```

```
[[0 1 1 1 0]
 [0 1 0 1 0]
 [0 1 1 1 0]
 [0 1 0 1 0]]
```



What is the number of independent features (columns) in this case?

ICA

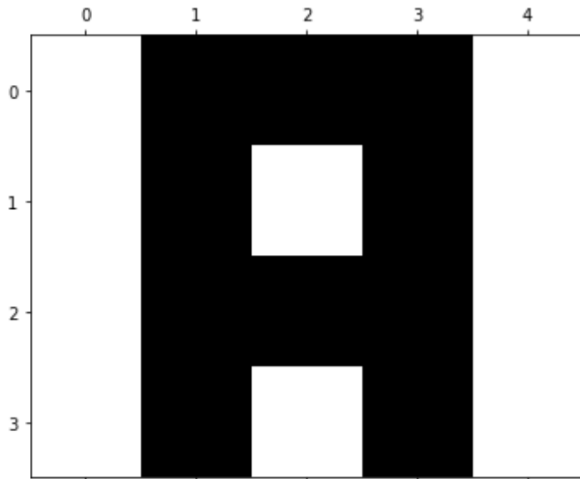
```
A = np.array([[0,1,1,1,0],
               [0,1,0,1,0],
               [0,1,1,1,0],
               [0,1,0,1,0]])

print(A)
```

```
plt.matshow(A, cmap='Greys')
plt.show()
```

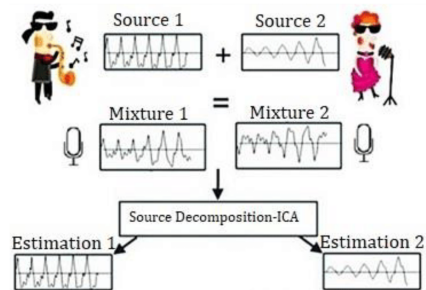
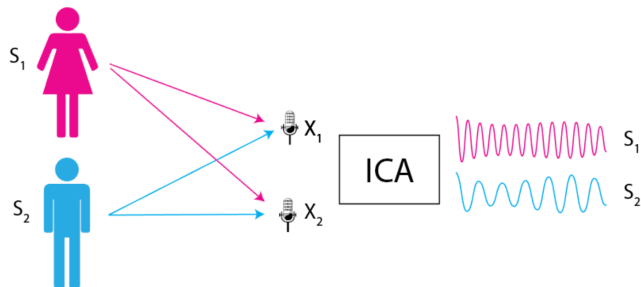
```
[[0 1 1 1 0]
 [0 1 0 1 0]
 [0 1 1 1 0]
 [0 1 0 1 0]]
```

```
[[0 1]
 [1 0]
 [0 1]
 [1 0]]
```



What is the number of independent features (columns) in this case? 2

Cocktail party problem



It refers to tracing and recognizing the speech of a specific speaker when multiple speakers talk simultaneously.

ICA - formal concept

Let's say we have a set of observations $x = \{x_1(t), x_2(t), \dots, x_n(t)\}$ where t is the index (time, for example). We assume these observations (signals) origin from some independent sources $s = \{s_1(t), s_2(t), \dots, s_n(t)\}$. Formally,

$$x = As$$

where A is a matrix. Then

$$s = A^{-1}x$$

or

$$s = Wx$$

where $W = A^{-1}$. The problem is we don't know both s and W so it cannot be solved in general case. However, under some reasonable assumptions and decompositions it is possible to get signal $\{s_i\}$. For example, if we assume $ss^T = I$, then

$$xx^T = (As)(As)^T = Ass^T A^T = AA^T$$

ICA - supplementary material

Optimization problem - example solutions:

- **Mutual information (I) minimization**

$$I(X, Y) = \sum_x \sum_y P(x, y) \log\left(\frac{P(x, y)}{P(x)P(y)}\right)$$

I is the measure of the mutual dependence between the two variables. More specifically, it quantifies the amount of information obtained about one random variable by observing the other random variable. If random variables X and Y are independent, then $P(x, y) = P(x)P(y)$ and $I(X, Y) = 0$.

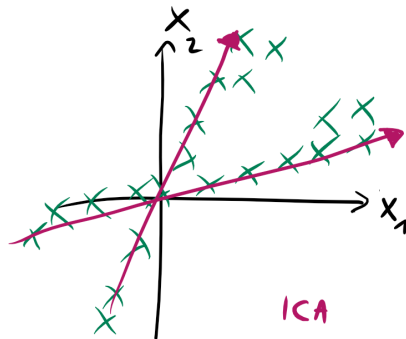
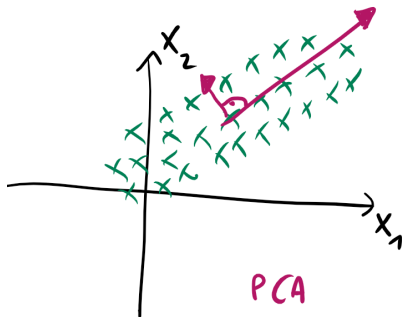
- **Non-Gaussianity maximization.** In practice, signal mixtures tend to have Gaussian probability density functions, and source signals tend to have non-Gaussian probability density functions. The process may be realized, for example, by maximizing the kurtosis which equals 3 for Gaussian distribution.

Before the process, we often make some preprocessing:

- centring,
- whitening (after this, the covariance matrix is the identity matrix),
- PCA algorithm at first.

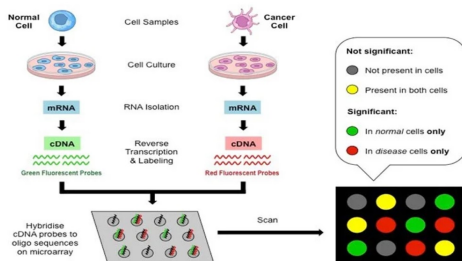
PCA vs ICA - comparison

- **PCA** - finds directions of maximal variance in gaussian data
- **ICA** - finds directions of maximal independence in nongaussian data



PCA vs ICA in microarray data

Microarray is a matrix of expression of genes in subsequent samples (arrays). Usually, samples correspond to different conditions.



APPLICATION OF INDEPENDENT COMPONENT ANALYSIS TO MICROARRAY DATA

Roland E. Suri
Intelligent Optical Systems
2520 W. 237th St., Torrance, CA 90505
(310) 530-7130
rsuri@intopsys.com

and
Computational Neurobiology Laboratory, Salk Institute¹
10010 North Torrey Pines Road, San Diego, CA 92037
(858) 453-4100

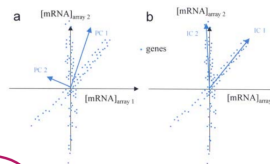


Figure 2. Comparison of PCA (left) with ICA (right) for hypothetical super-Gaussian data (normalized to a mean of zero) measured in two microarray experiments. ICA, but not PCA, finds the correct components. Each dot corresponds to the measured mRNA concentration of one gene. The mRNA concentration in condition 1 is shown on the x-axis and the mRNA concentration in condition 2 is shown on the y-axis. Such data is most likely the result of two coexpressed gene groups: one group is only regulated on array 2 (dots along the vertical axis); the other group is coexpressed for both arrays (dots along diagonal): (a) The PCA algorithm finds the orthogonal principal components PC 1 and PC 2 in the directions of the largest variations; (b) The ICA algorithm finds the independent components IC 1 and IC 2 that maximize the statistical independence of the data. Therefore, the IC components point in the direction of the gene groups that are independently expressed in both conditions, and are most likely to be regulated by independent processes.

	Array1	Array2	...	ArrayM
Gene1	value11	value21	...	valueM1
Gene2	value12	value22	...	valueM2
Gene3	value13	value23	...	valueM3
...	...	...	...	...
GeneN	value1N	value2N	...	valueMN

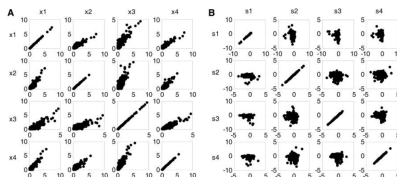
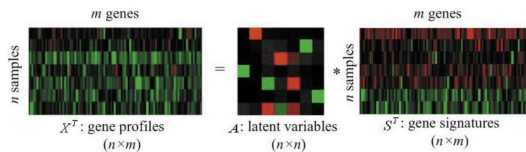
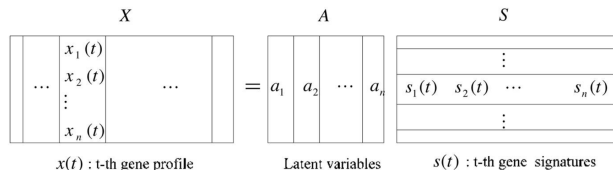
expression level
of the first gene
in Array1

Microarray experiment - case study

Go to **NCBI** (GEO DataSets) and find GDS810 record.

- What was investigated during the experiment? (Summary/Experiment design and value distribution)
- Apply k-means algorithm with $k = 2$ to genes. How many genes are in each group? (Cluster heatmaps)
- Apply hierarchical grouping to samples, where distance is measured by Pearson Correlation and linkage is UPGMA method. (Cluster heatmaps)
- Compare the results from previous exercise with samples annotations. (Experiment design and value distribution)

A review of independent component analysis application to microarray gene expression data, Biotechniques, 2010



(A) The joint distributions between four original severe Alzheimer's disease genes: x_1 , x_2 , x_3 , and x_4 . (B) The joint distributions between four corresponding gene signatures s_1 , s_2 , s_3 , and s_4 captured by FastICA.

A review of independent component analysis application to microarray gene expression data, Biotechniques, 2010

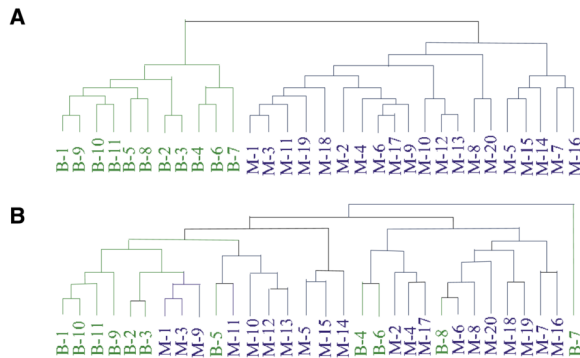


Figure 5. The superimposed hierarchical dendrogram in clustering benign (B1–B11) and malignant (M1–M20) endometrial samples by using ICA and PCA outputs in Saidi’s work (40) (A) ICA output with superimposed hierarchical cluster dendrogram. The hierarchical clustering was applied to the columns of matrix *A* and showed a distinct separation into benign (green lines) and malignant (blue lines) groups. (B) PCA output with superimposed hierarchical cluster dendrogram based on 15 principal components.

It's worth reading the full paper.

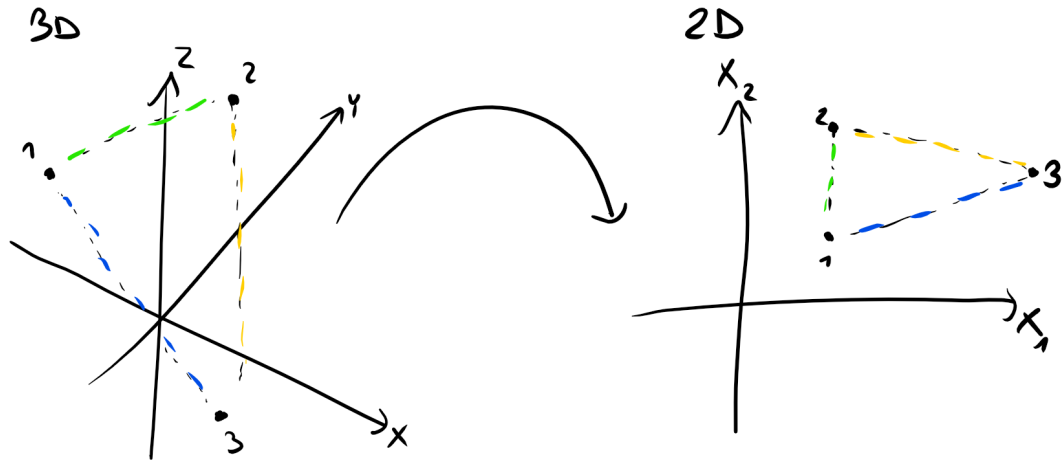
Scientific paper reading

Consider the paper **Metabolite fingerprinting: detecting biological features by independent component analysis**. Read at least the abstract and then, try to answer the following questions:

- What kind of data the authors considered? (DATA ACQUISITION - page2, Fig1/Fig2 - page3).
- What was the purpose of using PCA? (PCA—PRE-PROCESSING - page5).
- What are advantages of ICA over PCA? (ICA—INDEPENDENT COMPONENT ANALYSIS - page5).
- What was the purpose of using kurtosis? (SIGNIFICANT COMPONENTS – KURTOSIS - page5)
- What was the number of PCA components which provided the higher number of ICA components? (OPTIMAL PCA-DIMENSION - page6, Fig.7A - page7).
- What was the kurtosis of IC4? Was it interesting for further analysis? (Fig.7B - page8).

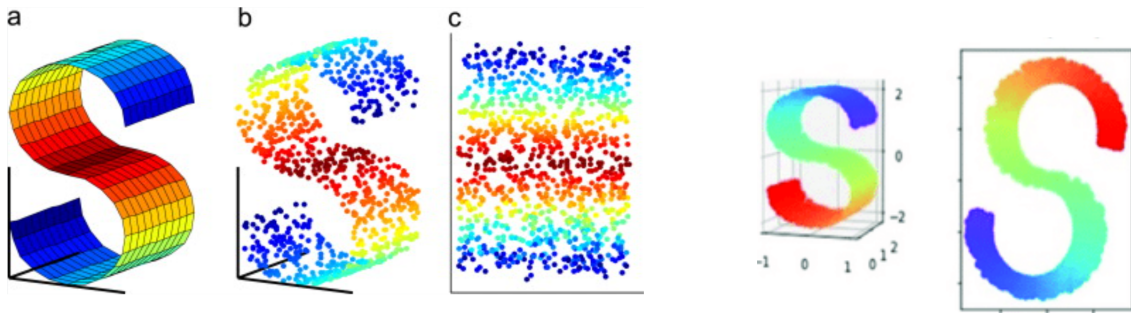
Methods based on distances

There is another large set of methods for reducing the dimensionality - based on distances.



The aim here is to preserve the distances between data points. Example methods: MDS (multidimensional scaling, t-SNE, UMAP).

Methods based on distances



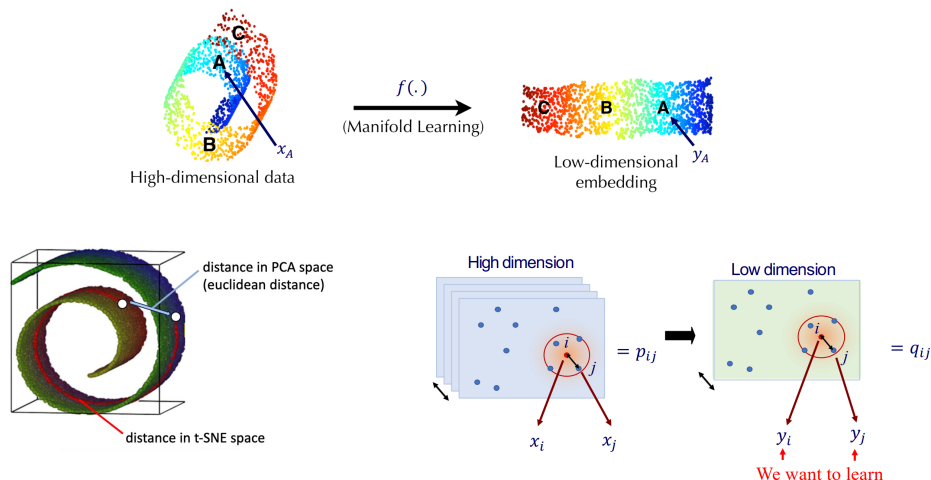
MDS, t-SNE, and UMAP are all dimensionality reduction methods based on distances. All of these methods use distances between points in the original space to reduce the dimensionality of the data and present it in a more accessible way. MDS relies on exact distances between points, while t-SNE and UMAP also take into account non-linear relationships between points. All of these methods aim to find a lower-dimensional representation of the data that preserves as much of the information contained in the original dataset as possible.

t-SNE vs UMAP

- t-SNE mainly preserves local similarity structure of data, not global similarity,
- UMAP constructs a high dimensional graph representation of the data and then optimizes a low-dimensional graph to be as structurally similar as possible.

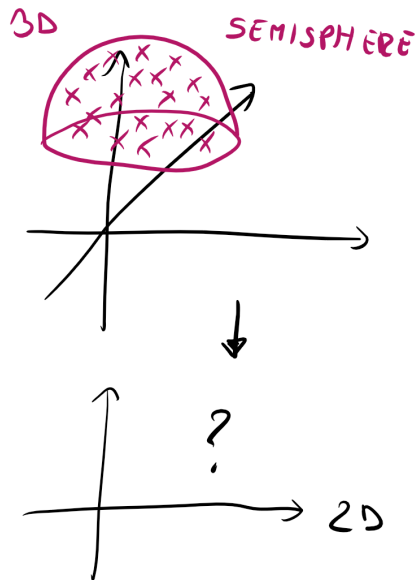
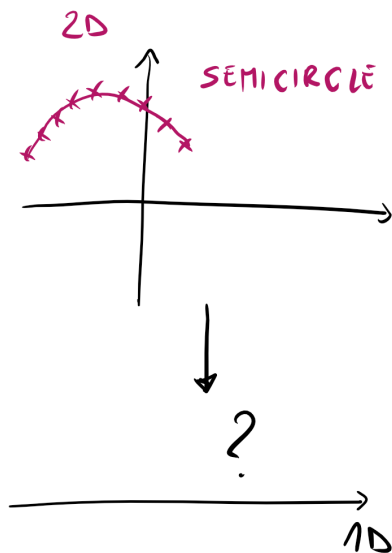
Methods based on distances

A manifold is a topological space that locally resembles Euclidean space near each point.

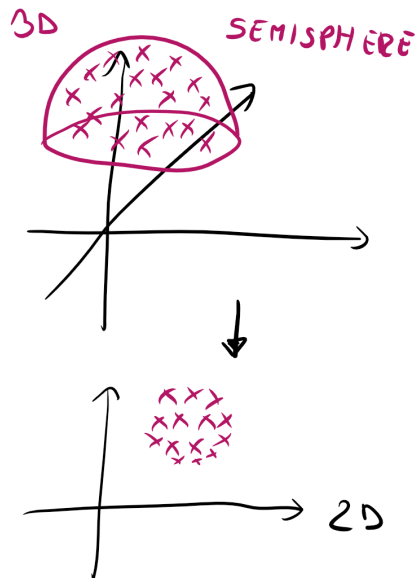
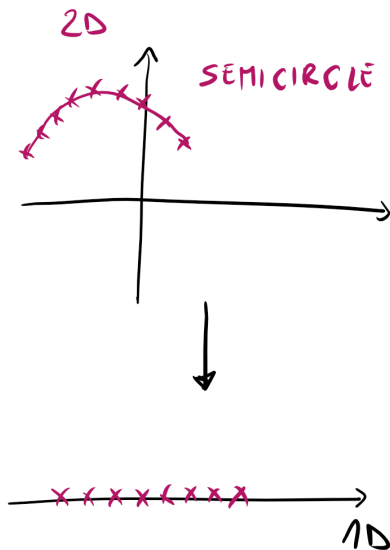


The aim is to map d dimensional manifold M ($\{x_i\}$ data) into a d' dimensional space ($\{y_i\}$ data), where $d' \ll d$.

Methods based on distances



Methods based on distances



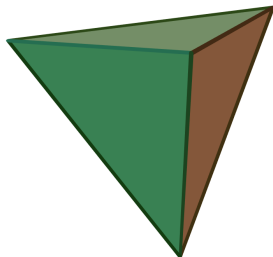
Exercise

How to find it? If $x_1, x_2, \dots, x_n$ are points in original space and $y_1, y_2, \dots, y_n$ are points in reduced space, define $d_{ij} = d(x_i, x_j)$ and $d_{ij}^* = d(y_i, y_j)$. We want to find $y_1, y_2, \dots, y_n$ which minimize

$$E = \sum_{i < j} (d_{ij}^* - d_{ij})^2$$

The task may be performed using Gradient Descent algorithm: $y_i^{new} = y_i^{old} - \alpha \frac{dE}{dy_i}$.

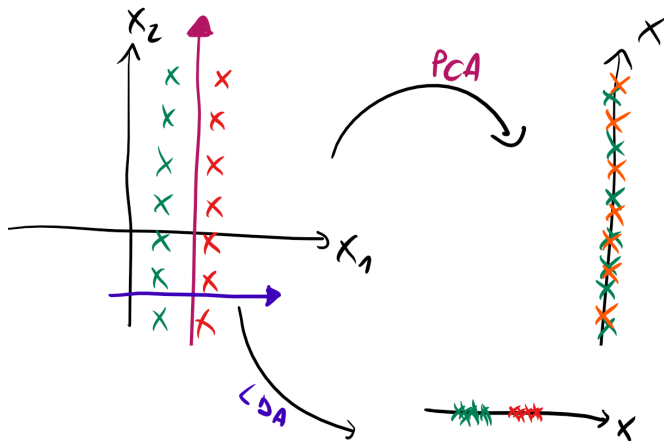
Exercise: Let's say we have four data points: $(1, 1, 1)$, $(-1, -1, 1)$, $(-1, 1, -1)$ and $(1, -1, -1)$ which are vertices of tetrahedron in 3D



- What is the distance between every pair of data points?
- Find 2D representation of this data.

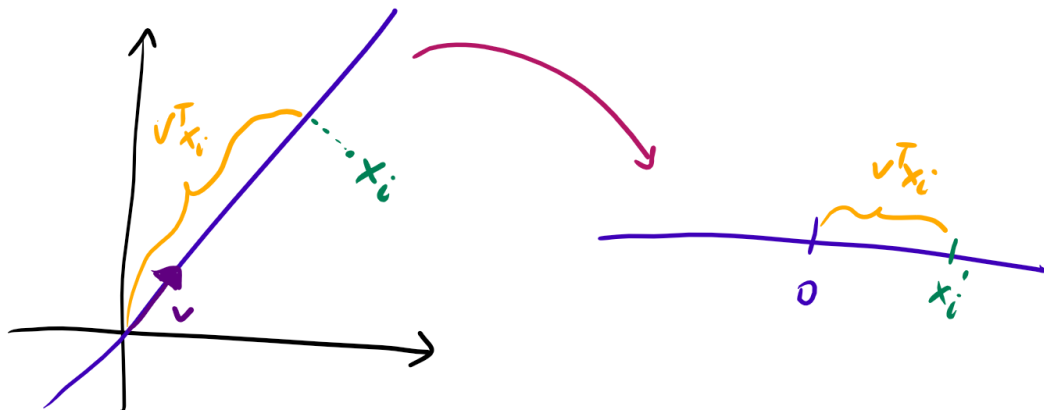
FDA (Fisher Linear Discriminant)

Fisher Linear Discriminant project to a line which preserves direction useful for data classification. Let's compare it with PCA.



Main idea: find projection to a line s.t. samples from different classes are well separated.

Projection



Let the line direction be given by unit vector v .

- Scalar $v^T x_i$ is the distance of projection of x_i from the origin.
- $v^T x_i$ is the projection of x_i into a one dimensional subspace.

FDA (Fisher Linear Discriminant)

Suppose we have 2 classes and d -dimensional samples $x_1, \dots, x_n$ where

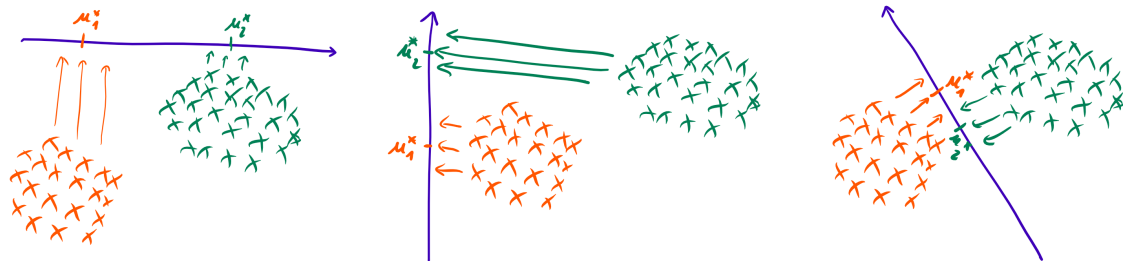
- n_1 samples come from the first class,
- n_2 samples come from the second class.

How to measure separation between projections of different classes?

Let μ_1^* and μ_2^* be the means of projection of classes 1 and 2. Then

$$\mu_1^* = \frac{1}{n_1} \sum_{x_i \in C_1} v^T x_i = v^T \left(\frac{1}{n_1} \sum_{x_i \in C_1} x_i \right) = v^T \mu_1.$$

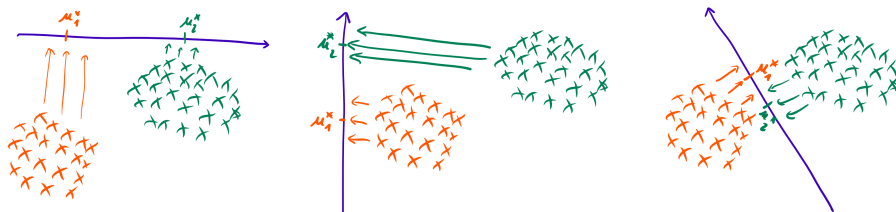
Similarly, $\mu_2^* = v^T \mu_2$.



Which direction separates the best?

FDA (Fisher Linear Discriminant)

The higher $|\mu_1^* - \mu_2^*|$ the better separation.



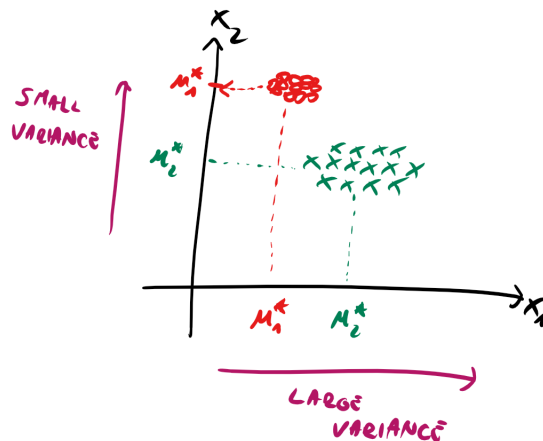
What about scatter (measure the same thing as variance)?

$$S = \sum_i (x_i - \mu)^2$$



Let $y_i = v^T x_i$. Then $s_1^* = \sum_{y_i \in C_1} (y_i - \mu_1^*)^2$, $s_2^* = \sum_{y_i \in C_2} (y_i - \mu_2^*)^2$.

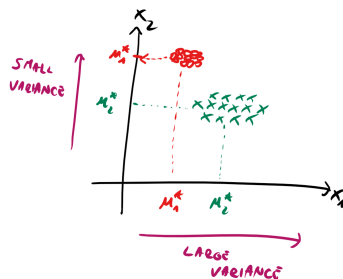
FDA (Fisher Linear Discriminant)



To sum up

- The higher $|\mu_1^* - \mu_2^*|$ the better separation.
- The smaller s_1^*, s_2^* the better separation.

FDA (Fisher Linear Discriminant)



The aim is to find the projection in the direction v that maximizes

$$J(v) = \frac{(\mu_1^* - \mu_2^*)^2}{s_1^* + s_2^*}$$

- We want projected means are far from each other.
- We want scatter in class 1/2 as small as possible (then, projected samples of class 1/2 are clustered around projected means).

FDA (Fisher Linear Discriminant)

$$J(v) = \frac{(\mu_1^* - \mu_2^*)^2}{s_1^* + s_2^*}$$

J may be expressed explicitly as a function of v .

$$J(v) = \frac{v^T S_B v}{v^T S_W v}$$

where $S_w = s_1 + s_2$ and $S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$ (class scatter matrix). By calculating the derivative and setting it to zero $\frac{dJ}{dv} = 0$ we may obtain the optimal v vector. It can be also approximated using Gradient Descent method

$$v = v + \alpha \frac{dJ}{dv}.$$

In general case, for more than two classes and projection matrix V we consider

$$J(V) = \frac{\det(V^T S_B V)}{\det(V^T S_W V)}.$$

Supplementary material

$$J(v) = \frac{(\mu_1^* - \mu_2^*)^2}{s_1^* + s_2^*}$$

$$\begin{aligned} s_1^* &= \sum_{y_i \in C_1} (y_i - \mu_1^*)^2 \\ &= \sum_{y_i \in C_1} (v^T x_i - v^T \mu_1)^2 \\ &= \sum_{y_i \in C_1} (v^T (x_i - \mu_1))^T (v^T (x_i - \mu_1)) \\ &= \sum_{y_i \in C_1} (v^T (x_i - \mu_1))^T ((x_i - \mu_1)^T v) \\ &= \sum_{y_i \in C_1} v^T ((x_i - \mu_1))^T (x_i - \mu_1) v \\ &= v^T s_1 v. \end{aligned}$$

Similarly, $s_2^* = v^T s_2 v$, so

$$s_1^* + s_2^* = v^T s_1 v + v^T s_2 v = v^T (s_1 + s_2) v.$$

And for projected means

$$(\mu_1^* - \mu_2^*)^2 = (v^T \mu_1 - v^T \mu_2)^2 = v^T (\mu_1 - \mu_2) (\mu_1 - \mu_2)^T v = v^T S_B v.$$

Finally

$$J(v) = \frac{v^T S_B v}{v^T S_W v}$$